



Physics-based Simulation of Deformable Objects with Deep Learning for Computer Graphics Applications

State of the Art Report

15/01/2026

Student : **Antonin GRANADOS**
antonin.granados@telecom-paris.fr

Supervisor : **Kiwon UM**
kiwon.um@telecom-paris.fr

School : **Télécom Paris**
Course : **MDC-4EL01-TP (RECH-201)**

Abstract

Abstract

Physics-based simulation of deformable objects, and cloth in particular, plays a central role in computer graphics but remains computationally demanding for applications requiring real-time or highly detailed motion. Recent advances in deep learning, and especially physics-informed Graph Neural Networks, offer a compelling alternative by learning deformation dynamics directly from synthetic simulation data. This report reviews the main methodologies underlying these approaches, with a focus on [MeshGraphNets \(MGN\)](#) and its subsequent extensions for improved long-range interaction handling, collision response, and temporal stability. We further examine the datasets and learning paradigms used to train these models, including supervised, semi-supervised, and self-supervised strategies grounded in physical constraints. Despite significant progress, several challenges persist: accumulated errors over long [rollouts](#), limited generalisation to unseen materials or [mesh](#) topologies, strong dependence on large high-quality synthetic datasets, and scalability issues for high-resolution garments. This survey analyses these limitations and outlines key research directions for enabling robust, efficient, and physically plausible cloth simulation in future graphics applications.

Contents

1. Physics-informed Learning	1
1.1. GraphNeuralNetwork (GNN) and Meshes	1
1.2. Core architecture	1
2. Dataset and Evaluation	3
2.1. Common Datasets	3
2.2. Self-supervised and Unsupervised Learning	3
3. Challenges and Limitations	4
3.1. Temporal stability and Generalisation	4
3.2. Dependence on large, high-quality Synthetic Datasets	5
3.3. Scalability to High-Resolution Meshes and Performance	5
Acronyms	6
Glossary	6
A Results Replication	7
A1. MeshGraphNets [9]	7
A2. Experiments With Alternative Learning Methods	7

Introduction

The simulation of deformable objects, especially cloth, is central to computer graphics applications. Over the years, realistic cloth behavior has been critical in film animation, video games, virtual reality, and more recently in virtual try-on for fashion. This problem has been researched for multiple decades, and a wide range of methods have been proposed to address it.

The most commonly used methods are physics-based [8, 7]. They rely on numerical solvers to integrate elastic energies and enforce bending or stretching constraints. While these methods can provide realistic motion, they are often computationally expensive. They require detailed objects and precise parameter fine-tuning, making them unusable in an interactive or real-time context and restricting them to offline computation.

But recently, deep learning approaches have emerged as a promising alternative. These methods leverage neural networks to learn the physical deformations of cloth from data. They promise orders of magnitude faster inference as well as strong generalisation capabilities, enabling their use in online applications and removing the need for complex and slow physics engines.

This report provides a comprehensive overview of the state of the art in physics-informed deep learning methods for cloth simulation. It covers key methodologies such as graph neural networks on [mesh](#) objects and physics-based loss functions, discusses usual datasets, and identifies the main technical bottlenecks that limit current performance.

1. Physics-informed Learning

1.1. [GraphNeuralNetwork \(GNN\)](#) and Meshes

3D [meshes](#) are fundamental in computer graphics applications, serving as a standard to represent 3D objects and scenes. This is because they are designed to be easily and efficiently processed by graphics cards. This inherent structure makes them ideal for simulations intended for computer graphics applications.

Various approaches have been explored to leverage [meshes](#) in deep learning models. One family of methods is presented in the following articles: [1, 12, 10, 11]. However, this is limited to virtual try-on applications and cannot be easily transposed to other simulations. The limitation comes from the fact that they use the [skeletal representation](#) of the body as the input to their model and predict the cloth deformation based on the body pose, which is finally [skinned](#) onto the model.

[GNNs](#) have become another popular choice for learning the physical dynamics of [mesh](#)-based structures, where the vertices of the object are modeled as nodes in a graph and edges encode local physical interactions. One of the most important approaches proposed is [MeshGraphNets \(MGN\)](#) [9]. Their models are not only able to simulate cloth dynamics but also a wide variety of physical phenomena, including fluid dynamics [6] and solid mechanics. The success of this architecture lies in its message-passing framework, where information propagates through the graph structure to model complex physical interactions.

1.2. Core architecture

A lot of the approaches that came after [MGN](#) [9] built upon its foundation, with many adapting its core architecture for specialized applications. These developments include improved handling of long-range dependencies @hood, better collision response [4], and enhanced temporal stability [5]. The

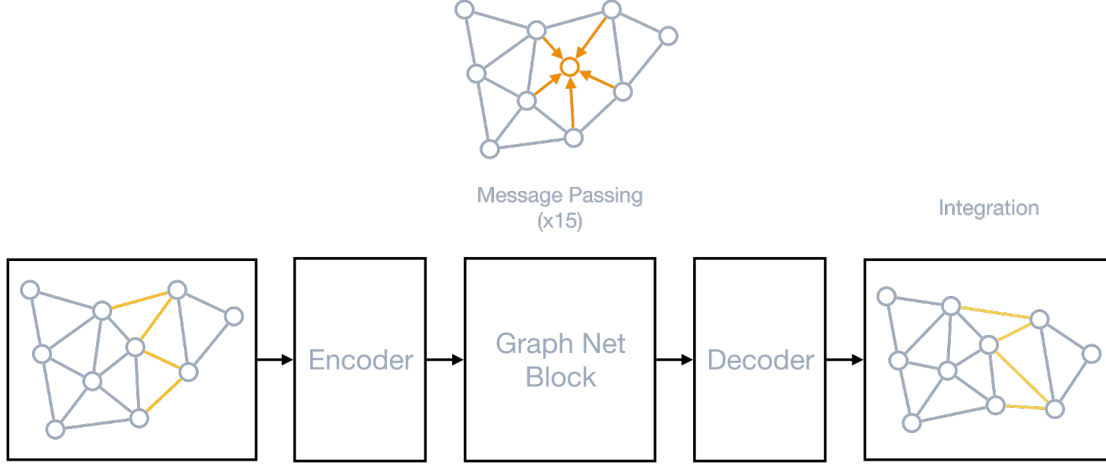


Figure 1: Representation of the **MGN** architecture, the yellow edges represent *world edges* and the grey edges are the *mesh edges* (adapted from **MGN** [9])

graph-based representation proves especially valuable for cloth simulation as it naturally accommodates irregular **mesh** structures and efficiently models local deformation patterns that are essential for realistic fabric behavior.

As in any **GNN** architecture, **MGN** represents a simulation state as a graph $G = (V, E)$, where V is the set of nodes (or vertices of the **mesh**) and E is the set of edges. This graph corresponds to the **mesh** of the object at a specific time step. Some architectures extend this representation [9, 4] (including **MGN**) by adding other types of edges, such as the so-called *world edges* (depicted in yellow in 1), which connect vertices that are close in space and help the message-passing process handle collisions.

The model follows an encoder-decoder architecture, where the node and edge features are first encoded to extract local information. They are then aggregated through multiple message-passing steps, where the features of each node are updated based on the features of its neighbors and the edges connecting them. Finally, the node features are decoded into accelerations to compute the deformation.

We use the notation e_{ij}^K to describe the feature vector of a the edge connecting vertex i to vertex j , and v_i is the feature vector of the vertex i . The superscripts M , W , and V respectively denote **mesh** edges, world edges, and vertices. And ε^K , f^K , δ^K are **multi-layer perceptron (MLP)**s. We can then formally described this by the following equations:

- Encoding:

$$e_{ij}^M \leftarrow \varepsilon^M(e_{ij}^M), \quad e_{ij}^W \leftarrow \varepsilon^W(e_{ij}^W), \quad v_i \leftarrow \varepsilon^V(v_i)$$

- Message Passing:

$$e'_{ij}^M \leftarrow f^M(e_{ij}^M, v_i, v_j), \quad e'_{ij}^W \leftarrow f^W(e_{ij}^W, v_i, v_j), \quad v'_i \leftarrow f^V(v_i, \sum_j e'_{ij}^M, \sum_j e'_{ij}^W)$$

- Decoding:

$$e_{ij}^M \leftarrow \delta^M(e'_{ij}^M), \quad e_{ij}^W \leftarrow \delta^W(e'_{ij}^W), \quad v_i \leftarrow \delta^V(v'_i)$$

2. Dataset and Evaluation

2.1. Common Datasets



Figure 2: Frame from two different datasets

Learning cloth simulation models requires large datasets capturing a wide diversity of fabrics, motions, geometries, and collision events. Since collecting real-world ground-truth cloth motion is extremely difficult, most datasets rely on synthetic generation using physics-based simulators.

Most papers here use synthetic data generated using the *ARCSim* simulator [8, 7]. This simulation engine allows for the generation of many deformable materials such as cloth, paper, plastic, and metal. It also supports remeshing, which can help with the creation of realistic behaviors. The original *MGN* [9] used this algorithm in combination with their architecture to improve the quality of the predicted cloth, but this comes with a significant computational cost and was not used in subsequent works.

Articles focusing on *virtual try-on (VTO)* applications such as [12, 10] rely on more advanced pipelines. In their case, they use *motion capture (MoCap)* data (from the *CMU Graphics Lab Motion Capture Database*), which provides a skeletal representation of a human body. These motions are then converted to the *SMPL* (Skinned Multi-Person Linear) format to obtain a human *mesh*. Finally, they use *ARCSim* to generate synthetic cloth draped on the bodies.

2.2. Self-supervised and Unsupervised Learning

Different learning frameworks have been proposed to tackle the cloth simulation problem. The first one used in the original *MGN* paper [9] was a supervised learning approach that leveraged precomputed ground-truth cloth motion generated by the simulator. Their implementation uses a loss computed as the error between the predicted and ground-truth cloth meshes, where the predicted *mesh* is obtained by integrating the predicted vertex accelerations with a forward Euler step on positions. In practice this is a per-node L_2 loss on decoder outputs (accelerations in our case) against simulator targets, sometimes with an additional L_2 term on the sizing field when learned remeshing is enabled. Training

is performed on one-step targets but evaluated on long [rollouts](#); some works also train on short [auto-regressive rollout](#) segments to reduce exposure bias and improve long-horizon stability.

More advanced methods, such as [5], use physics-based information. Which is semi-supervised learning, meaning that they add more physical constraints both to the loss functions and to the [mesh](#) features. For instance, they track the kinetic energy of each node and use it as an additional feature. They also incorporate this quantity into the loss to encourage energy conservation, which helps improve temporal stability. A typical formulation is

$$L = L_{sup} + \lambda E_{quad} \quad \text{with} \quad E_{quad} = \sum_i (E_i - \text{dash}(E))^2$$

where L_{sup} is the per-node L_2 target on accelerations, E_i is the kinetic energy of node i (or its change), and $\text{dash}(E)$ is a target or running estimate. The supervised weight can be decayed over time so that the physics term dominates late training.

Finally, the last family of approaches relies on [self-supervised](#) learning. Instead of using ground-truth data in the loss, the model is trained using physics-inspired loss functions that enforce realistic cloth behavior. SNUG [11] is a representative example: it learns dynamic garment motion purely from geometric constraints such as stretching, bending, and temporal consistency. Concretely, the loss is split as

$$L = L_{inertia} + L_{static}$$

with

$$\begin{aligned} L_{inertia} &= 1/2 \Delta t^2 (x - \hat{x})^\top M (x - \hat{x}) \quad \text{and} \\ L_{static} &= L_{strain} + L_{bending} + L_{gravity} + L_{collision} \end{aligned} \tag{1}$$

with $\hat{x} = x + v \Delta t$ the explicit Euler prediction from the previous step.

The strain term comes from the Saint Venant-Kirchhoff material with Lamé parameters (λ, μ) , bending penalizes the squared dihedral angle, gravity is the vertex potential energy $-mg^\top x$, and collision is a cubic penalty $k_{col} \max(\varepsilon - d(x), 0)^3$. The weights correspond to physical parameters, avoiding heuristic loss balancing. Similar ideas appear in several works [1, 3, 2], which incorporate collision penalties, regularization terms, or local rigidity constraints to guide learning without explicit supervision. SENC [4] further augments the loss with explicit self-collision penalties derived from proximity-based world edges to improve cloth–cloth contact.

Training in these self-supervised settings is often performed on short [rollouts](#) (e.g., 3-frame subsequences in @snug) to stabilise recurrent hidden states and better align the loss with [rollout](#) behavior.

3. Challenges and Limitations

3.1. Temporal stability and Generalisation

Most approaches discussed here are [auto-regressive](#), as they predict each frame of the cloth motion from the previous ones. But the issue that arises from this method is that errors in the prediction of the model tend to accumulate over a [rollout](#). While the results are visually plausible because they do not explode (i.e., the [mesh](#) keeps a reasonable scale), they can still drift away from the ground truth when using supervised learning. This may indicate physical accuracy limitations of the ground-truth simulation (see @dataset-limits), or that the model is simply not precise enough. This problem is less noticeable when using [self-supervised](#) learning because physical meaning is incorporated into the training.

Another challenge is the generalisation of the model to unseen cloth materials or meshes. The methods that use [skinning](#) such as @neural-cloth-simulation [12, 10, 11] usually cannot generalise to

unseen structures because the model size is directly dependent on the node count. In this case, they are only able to predict new poses. Methods that rely on [GNNs](#) do not have this problem, as the network is not dependent on the cloth shape and directly learns local interactions. However, these interactions are tied to the material type, and changing the material typically requires retraining the model. Material descriptions could be incorporated into the node or edge features to help generalisation. For instance, using Lamé parameters or bending/stretching stiffness as features could help the generalisation ability of the model.

3.2. Dependence on large, high-quality Synthetic Datasets

In a supervised training setup, a large and high-quality dataset is required to train the model. This usually means that simulations taking days to compute are necessary. The quality of the dataset is extremely important because it is directly used as the target output, and any physical inaccuracies will be propagated to the model. This is why some methods use semi-supervised learning: they use the ground truth as a reference but add a physical loss to avoid learning the simulator’s errors.

Even when using [self-supervised](#) learning, the training depends on large datasets. This ground truth material is not used in the loss function, but it is needed to provide realistic input to the model. The quality of the dataset in this case is not necessarily as high as in supervised learning, but it still needs to be physically plausible.

An idea to improve on these methods would be to use semi-supervised learning with a decaying factor applied to the ground truth error. This would make the model learn faster in the beginning by trying to mimic the simulation, but by the end of the training it would minimize only the physical loss and would not be influenced by the simulation’s inaccuracies.

3.3. Scalability to High-Resolution Meshes and Performance

A key limitation of current [GNN](#)-based cloth simulation methods is their difficulty scaling to high-resolution [meshes](#). As the number of vertices grows, the number of edges increases proportionally, which significantly raises the computational cost of message passing. This is especially problematic for dense garments or high-quality assets commonly used in film or offline rendering pipelines. Even when the network size remains constant, the underlying graph becomes too large to process efficiently on a single GPU, resulting in slow inference or memory overflows.

Some approaches attempt to solve this problem by relying on multiscale or hierarchical representations, where the simulation is run on a coarse structure and then upsampled as explored in HOOD [2]. While this reduces computation, it also limits the accuracy of fine wrinkles or high-frequency deformations unless sophisticated refinement strategies are introduced. Other methods such as FastClothGNN [13] try to sparsify the connectivity, but this can discard important local interactions necessary for accurate cloth behavior.

Performance is also affected by the fact that message passing is not trivially parallelizable over multiple steps. Unlike classical physics simulators, where constraints can be processed in optimized batches on the GPU, [GNN](#) operations involve scattered memory accesses and irregular graph structures. This makes real-time performance difficult to achieve when the [mesh](#) resolution becomes too large. For this reason, existing models typically rely on moderately sized [meshes](#), which restricts the visual fidelity and realism of the predicted deformations.

Conclusion

In this report, we reviewed recent advances in physics-informed deep learning techniques for cloth simulation. Modern approaches increasingly rely on graph neural networks to model local physical interactions on mesh structures, and several extensions of the [MGN](#) architecture have demonstrated strong performance across a range of dynamic phenomena. At the same time, new learning paradigms such as [self-supervised](#) and physics-driven losses show promise in reducing reliance on explicit ground-truth supervision and improving the physical plausibility of predictions.

Despite these advances, several challenges remain. Current models still struggle with long-term temporal stability, and their ability to generalise to unseen materials or geometries is limited. Training also depends heavily on the availability of large, high-quality synthetic datasets, which are costly to generate and may contain inaccuracies that propagate into the learned models. Finally, [GNN](#)-based simulators face computational constraints when scaling to high-resolution meshes, restricting their use in settings that require fine-grained detail or real-time performance.

Overall, physics-informed deep learning has shown significant potential for accelerating cloth simulation while maintaining realism, but further progress is needed to address these limitations. Improving generalisation capabilities, reducing dataset dependence, and enabling efficient computation on dense meshes remain key directions for future research.

Acronyms

GNN GraphNeuralNetwork. [1](#), [2](#), [5](#), [6](#)

MGN MeshGraphNets. [1–3](#), [6](#), [7](#)

VTO virtual try-on. [3](#)

Glossary

auto-regressive Auto-regressive models are models that predict future values based on past values. [4](#)

mesh In 3D computer graphics, a (polygon) mesh is a set of vertices, edges and faces that form the surface of an object. The faces consist of convex polygons, usually triangles for easy processing. [1–5](#), [7](#)

motion capture (MoCap) Motion Capture is the process of recording and analyzing human motion data. [3](#)

multi-layer perceptron (MLP) A multi-layer perceptron is a neural network with multiple layers of interconnected neurons. [2](#)

rollout In learning-based physical simulation, a rollout is the process of repeatedly applying a model over successive time steps to generate a full simulated trajectory. [1](#), [4](#), [7](#)

self-supervised Self-supervised learning is a paradigm where supervision signals are derived automatically from the data itself, without requiring explicit ground-truth labels. [4–6](#)

skeletal representation In computer animation, 3D models such as human body are represented as a [mesh](#) and a skeleton. The skeleton is a set of bones and joints which are connected together and influence the [mesh](#) position. 1

skinning Skinning is the process of putting a [mesh](#) on top of a skeleton. 1, 4

A Results Replication

A1. [MeshGraphNets](#) [9]

To evaluate the performance of [MGN](#), we implemented a focused replication of the cloth-related experiments from the original paper [9]. Our replication targets only deformable cloth objects, as these are directly aligned with the goals of the report. Other types of simulations, such as fluid dynamics or rigid body dynamics, are beyond the scope of this report.

The model follows the architecture, including the encoder–processor–decoder structure. In contrast to the original implementation, we did not include the adaptive remeshing component. This decision was driven both by its complexity and by the fact that later works [MGN](#) typically avoid remeshing while still achieving realistic cloth predictions.

In a first experiment, we did not use self-collision or collision with objects. With this restriction, we only needed world edges. Later on, we tried to include collisions, but the training process becomes much longer to achieve satisfactory results.

Training data was generated using a custom Python interface to ARCSim [8, 7], allowing us to create controlled cloth simulations with varying parameters and motion conditions. These datasets are smaller than those used in [9], but they capture sufficient dynamic variability to train and evaluate the model. Under these conditions, the replicated [MGN](#) implementation successfully learns to produce visually plausible cloth trajectories. However, as expected, the model exhibits drift during long [rollout](#) and struggles to reproduce fine-scale wrinkles. Despite these limitations, the replication provides a reliable baseline for further experimentation.

A2. Experiments With Alternative Learning Methods

Beyond the supervised setup used in [MGN](#), we explored several alternative learning strategies, including semi-supervised and self-supervised formulations. These approaches aim to reduce the reliance on high-quality ground-truth datasets and to embed additional physical meaning directly into the training objective.

Our semi-supervised experiments combine a standard supervised loss with physics-inspired regularization terms, such as the ones described in HOOD @hood and SNUG @snug.

References

- [1] Hugo Bertiche, Meysam Madadi, and Sergio Escalera. “Neural Cloth Simulation”. In: (2022). DOI: [10.1145/3550454.3555491](https://doi.org/10.1145/3550454.3555491). arXiv: [2212.11220](https://arxiv.org/abs/2212.11220).
- [2] Artur Grigorev et al. “HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics”. In: (2022). arXiv: [2212.07242](https://arxiv.org/abs/2212.07242).
- [3] Yudi Li et al. “N-Cloth: Predicting 3D Cloth Deformation with Mesh-Based Networks”. In: *Computer Graphics Forum (Proceedings of Eurographics)* 41.2 (May 2022), pp. 547–558.
- [4] Zhouyingcheng Liao, Sinan Wang, and Taku Komura. *SENC: Handling Self-collision in Neural Cloth Simulation*. 2024. arXiv: [2407.12479](https://arxiv.org/abs/2407.12479).
- [5] Emmanuel Ian Libao et al. “MeshGraphNetRP: Improving Generalization of GNN-based Cloth Simulation”. In: *ACM SIGGRAPH Conference on Motion Interaction and Games*. MIG ’23. ACM, Nov. 2023, pp. 1–7. DOI: [10.1145/3623264.3624441](https://doi.org/10.1145/3623264.3624441). URL: <http://dx.doi.org/10.1145/3623264.3624441>.
- [6] Mohammad Amin Nabian et al. “X-MeshGraphNet: Scalable Multi-Scale Graph Neural Networks for Physics Simulation”. In: *ACM SIGGRAPH Conference on Motion Interaction and Games*. MIG ’23. ACM, Nov. 2024, pp. 1–7. DOI: [10.1145/3623264.3624441](https://doi.org/10.1145/3623264.3624441). URL: <http://dx.doi.org/10.1145/3623264.3624441>.
- [7] Rahul Narain, Tobias Pfaff, and James F. O’Brien. “Folding and Crumpling Adaptive Sheets”. In: *ACM Transactions on Graphics* 32.4 (July 2013). Proceedings of ACM SIGGRAPH 2013, Anaheim, 51:1–8. URL: <http://graphics.berkeley.edu/papers/Narain-FCA-2013-07/>.
- [8] Rahul Narain, Armin Samii, and James F. O’Brien. “Adaptive Anisotropic Remeshing for Cloth Simulation”. In: *ACM Transactions on Graphics* 31.6 (Nov. 2012). Proceedings of ACM SIGGRAPH Asia 2012, Singapore, 147:1–10. URL: <http://graphics.berkeley.edu/papers/Narain-AAR-2012-11/>.
- [9] Tobias Pfaff et al. “Learning Mesh-Based Simulation with Graph Networks”. In: (2021). arXiv: [2010.03409](https://arxiv.org/abs/2010.03409) [cs.LG]. URL: <https://arxiv.org/abs/2010.03409>.
- [10] Igor Santesteban, Miguel A. Otaduy, and Dan Casas. “Learning-Based Animation of Clothing for Virtual Try-On”. In: *Computer Graphics Forum (Proc. Eurographics)* (2019). ISSN: 1467-8659. DOI: [10.1111/cgf.13643](https://doi.org/10.1111/cgf.13643).
- [11] Igor Santesteban, Miguel A. Otaduy, and Dan Casas. *SNUG: Self-Supervised Neural Dynamic Garments*. 2022. arXiv: [2204.02219](https://arxiv.org/abs/2204.02219).
- [12] Igor Santesteban et al. “Self-Supervised Collision Handling via Generative 3D Garment Models for Virtual Try-On”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021).
- [13] Yang Zhang, Kailuo Yu, and Xinyu Zhang. “FastClothGNN: Optimizing message passing in Graph Neural Networks for accelerating real-time cloth simulation”. In: *Graphical Models* 139 (2025), p. 101273. ISSN: 1524-0703. DOI: <https://doi.org/10.1016/j.gmod.2025.101273>. URL: <https://www.sciencedirect.com/science/article/pii/S1524070325000207>.